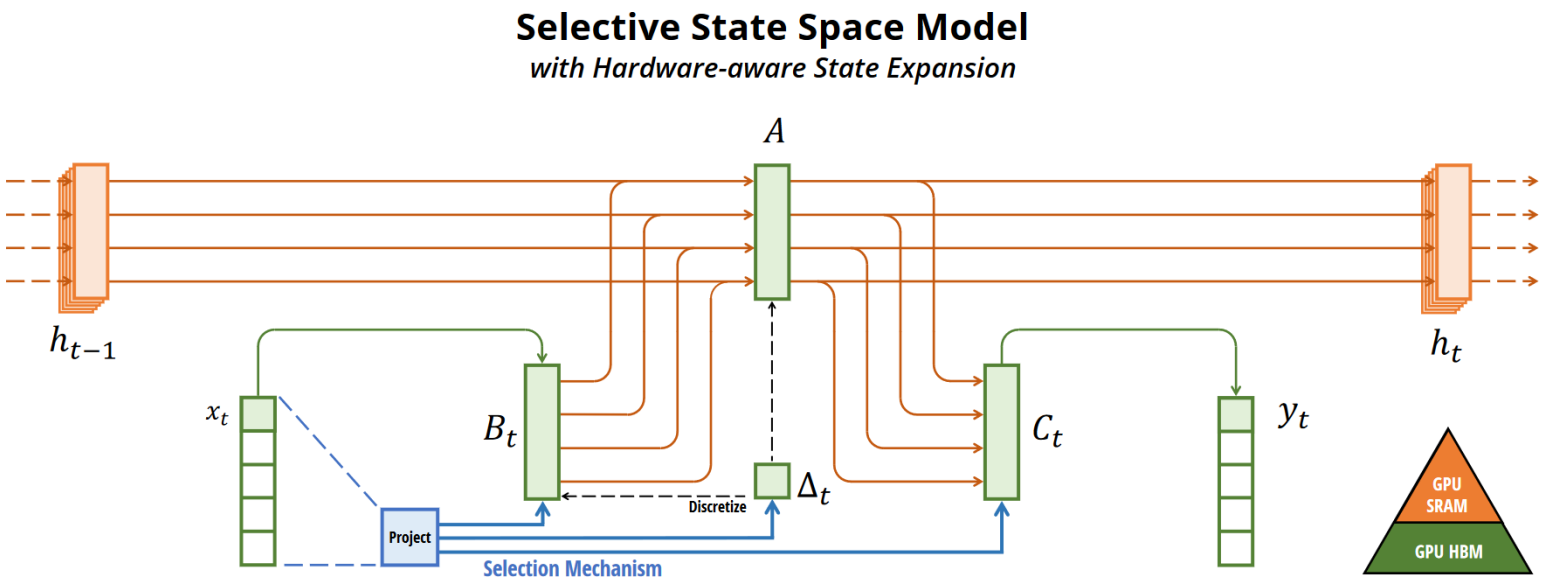


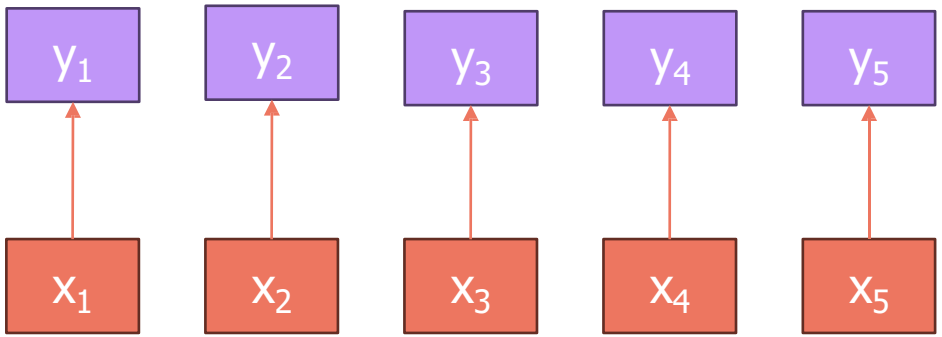
Mamba: Linear-Time Sequence Modeling with Selective State Spaces

Graph SSM

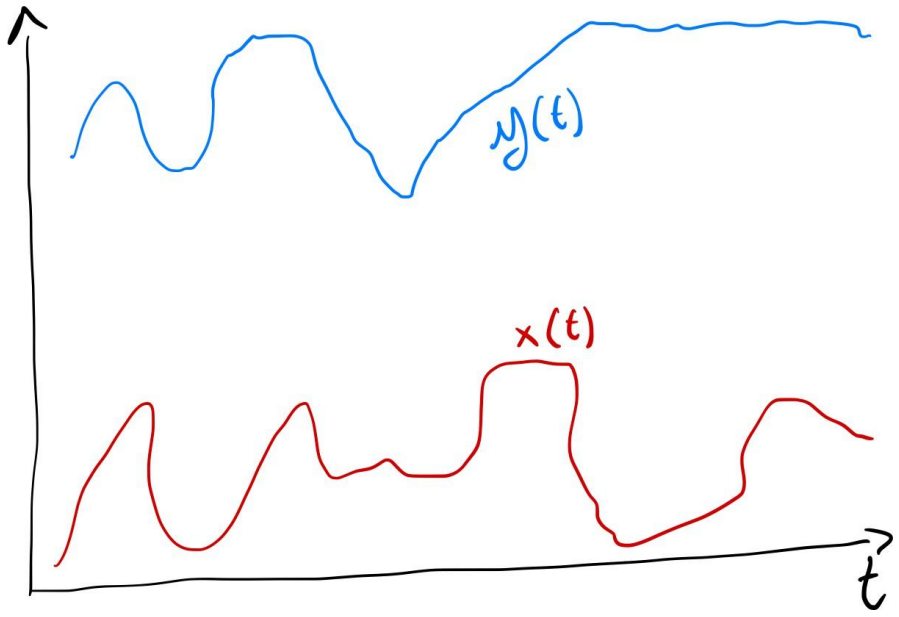
Graph Mining – C3 Evaluation

Sequence modeling

The goal of a sequence model is to map an input sequence, to an output sequence. We can map a continuous input signal $x(t)$ to an output signal $y(t)$ or a discrete input sequence to a discrete output sequence.



Discrete signal



Continuous signal

Let's review the existing models for sequence modelling.

Sequence modeling: RNN

Theoretically with infinite context window

Practically suffer from vanishing/exploding gradient

Training: $O(N)$, **Not Parallelizable because to generate the nth token, we need the (n-1)th token.**

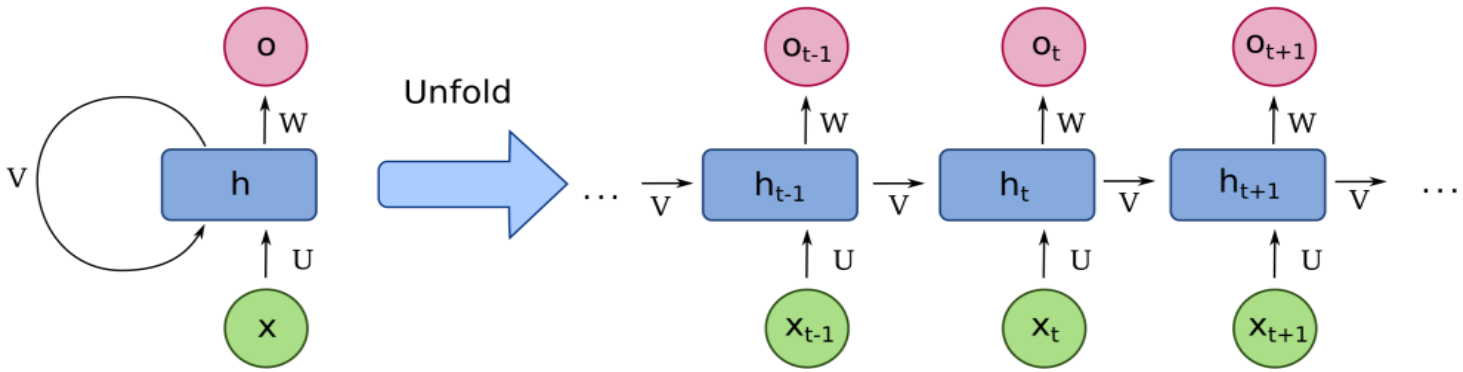
Inference: Constant time for each token, which means the computation required to produce first token is same as producing the 10th token ….

Apply a **recurrence relation** at every time step to process a sequence:

$$\boxed{h_t} = \boxed{f_W}(\boxed{x_t}, \boxed{h_{t-1}})$$

cell state — function with weights W — input — old state

RNN



Sequence modeling: Transformers

Finite context window

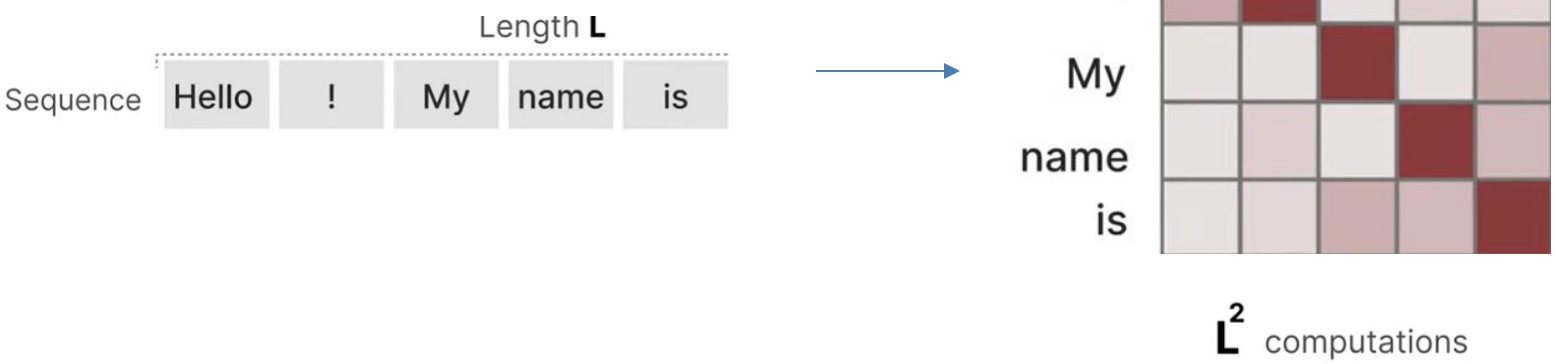
Training: $O(N^2)$, **Easily Parallelizable**

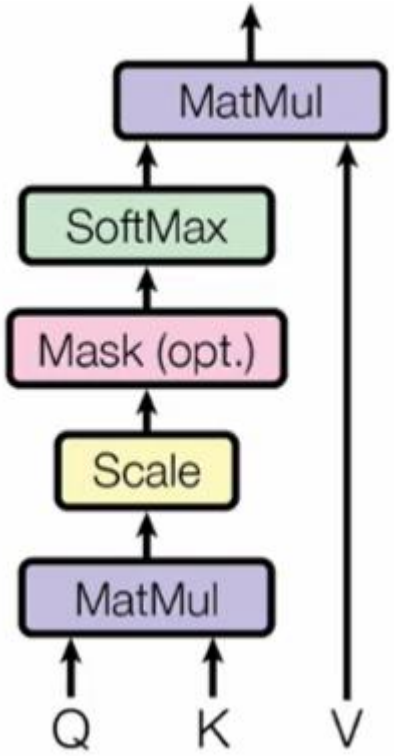
Inference: For each token. This means that if we want to generate the 10th token, we need to perform 10 dot product. To generate the 11th token, we will need to perform 11 dot products, etc.

Given, we have a sequence of length L we need quadratically expensive compututaion

Training can be parallelized due to attention

$$\text{Attention}(\text{Q, K, V}) = \text{Softmax}\left(\frac{QK^\top}{\sqrt{d_k}}\right)V$$





Transformers Architecture

Sequence modeling: SSM

In an ideal world, we would like a model for which we can:

1. Parallelize the training (like the Transformer) and can scale linearly to long sequences (with a computation/memory cost of O(N) like the RNN)

1. Can inference each token with a constant computation/memory cost (O(1) like the RNN)…

Let's explore State Space Models (SSM)… but first, we need to review some maths.

Introduction to **differential** equations

Let's talk about differential equations with a very simple example. Imagine you have some bunnies, and the population grows at a constant

rate of λ proportional to the number of bunnies, which means that every bunny will give birth to λ baby bunnies. So, we can say that the rate of change of the population of bunnies is the following:

$$(\text{number of baby bunnies born on a particular time step } t) = \lambda \times (\text{bunnies on the same time step } t)$$

$$(\text{rate of change w.r.t time}) = \lambda \times (\text{bunnies at time t})$$

$$\frac{db}{dt} = \lambda b(t)$$

$\frac{db}{dt}$ indicates the rate of change of the population of bunnies w.r.t time

$b(t)$ indicates how many bunnies we have at time t

We can clearly verify that the solution is $b(t) = ke^{\lambda t}$

where $k = b(0) = 5$, let's assume that is our initial population of bunnies.

We usually use differential equations to model the state of a system over time, with the goal of finding a function that gives us the state of the system at any time step, given the initial state of the system at time 0.

State space models

A state space model allows us to map an input signal $x(t)$ to an output signal $y(t)$ by means of a state representation $h(t)$ as follows:

$$h^{'}(t) = \boldsymbol{A}h(t) + \boldsymbol{B}x(t)$$
$$y(t) = \boldsymbol{C}h(t) + \boldsymbol{D}x(t)$$

This state space model is **linear** and **time invariant**.

Linear because the relationships in the expressions above are linear, and time invariant because the parameter matrices **A**, **B**, **C**, **D** do not depend on time (they are fixed).

Note: For now consider A, B, C, D, x(t), h(t) and y(t) to be numbers, not vectors.

Later we will extend our analysis to vectors.

To find the output signal $y(t)$ at time t, we first need to find a function $h(t)$ that describes the state of the system for all time steps. But that can be hard to solve analytically.
Usually we never work with continuous signals, but always with discrete ones, so how can we produce outputs $y(t)$ for a discrete signal?

We first need to discretize our system!

Discretization

To solve a differential equation we need to find the function h(t) that makes the two hand sides of the equation equal, but most of the time it's hard to find the analytical solution of a differential equation, that's why we can approximate the solution of a differential equation. To find the approximate solution of a differential equation means to find a sequence of h(0), h(1), h(2), h(3), etc. that describe the evolution of our system over time. So instead of finding $h(t)$ we want to find $h(t_k) = h(k\Delta)$ where Δ is our step size.

Remember the bunnies problem? Let's try to find the approximate solution - which I remind you is a function $b(t)$ - using **Euler's method**!

1. First let's re-write our bunnies population model: $b'(t) = \boldsymbol{\lambda}\mathrm{b(t)}$
2. The derivative of a function is the rate of change of the function, that is: $\lim_{\Delta \to 0} \frac{b(t+\Delta)-b(t)}{\Delta} = b'(t)$. So, by choosing a small step size Δ we can get rid of the limit: $\frac{b(t+\Delta)-b(t)}{\Delta} \cong b'(t)$. By multiplying with Δ and moving terms around we can further write: $b(t+\Delta) \cong b'(t)\Delta + b(t)$
3. We can then plug the bunnies population model into the previous expression to obtain: $b(t+\Delta) \cong \boldsymbol{\lambda}\mathrm{b(t)}\Delta + b(t)$
4. We obtained a **recurrent** formulation!

Discretization

By using a similar reasoning to the bunny scenario, we can also discretize our state space model, so that we can calculate the evolution of the state over time by using a **recurrent** formulation.

1. By using the definition of derivative we know that: $h(t+\Delta) \cong \Delta h'(t) + h(t)$
2. This is the (continuous) state space model: $h'(t) = \boldsymbol{A}h(t) + \boldsymbol{B}x(t)$
3. We can substitute the state space model into the first expression to get the following

$$
\begin{aligned}
h(t+\Delta) &\cong \Delta\big(\boldsymbol{A}h(t) + \boldsymbol{B}x(t)\big) + h(t) \\
&= \Delta\boldsymbol{A}h(t) + \Delta\boldsymbol{B}x(t) + h(t) \\
&= (I + \Delta\boldsymbol{A})h(t) + \Delta\boldsymbol{B}x(t) \\
&= \overline{\boldsymbol{A}}h(t) + \overline{\boldsymbol{B}}x(t)
\end{aligned}
$$

Discretization

We have a recurrent formula that allows us to iteratively calculate the state of the system one step at a time, knowing the state at the previous time step.

The matrices $\boldsymbol{A}$ and $\boldsymbol{B}$ are the discretized parameters of the model.

This allows us to also calculate the output $y(t)$ of the system for discrete time steps

$$h'(t) = \boldsymbol{A}h(t) + \boldsymbol{B}x(t) \quad (1a) \qquad h_t = \overline{\boldsymbol{A}}h_{t-1} + \overline{\boldsymbol{B}}x_t \quad (2a)$$

$$y(t) = \boldsymbol{C}h(t) \quad (1b) \qquad y_t = \boldsymbol{C}h_t \quad (2b)$$

We do not choose the discretization step Δ, but we make it a parameter of the model so that it can be learnt with gradient descent

Recurrent computation

$$h_t = \overline{A}h_{t-1} + \overline{B}x_t$$
$$y_t = Ch_t$$

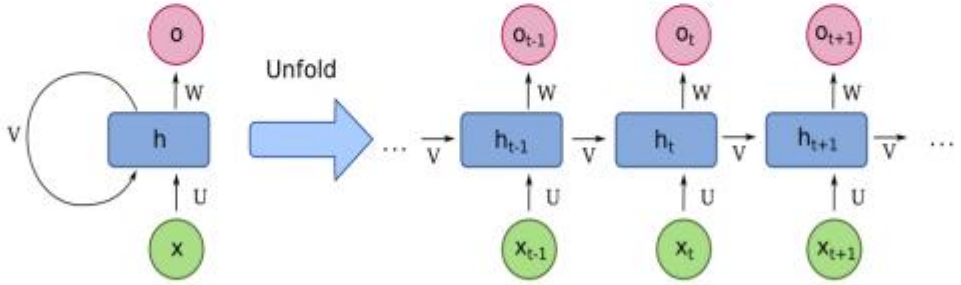
Now that we have our recurrent formula, how can we use it to calculate the output of the system for various time steps?

Suppose that, for simplicity, the initial state of the system is 0.

$$h_0 = \bar{B}x_0$$
$$y_0 = Ch_0$$

$$h_1 = \bar{A}h_0 + \bar{B}x_1$$
$$y_1 = Ch_1$$

$$h_2 = \bar{A}h_1 + \bar{B}x_2$$
$$y_2 = Ch_2$$



Source: Wikipedia

The Recurrent formulation is not good for training, because during training we already have all the tokens of the input and the target, so we want to parallelize the computation as much as possible.

So, SSM provides a convolutional mode as well.

Convolutional computation

Let's expand the output we built before for every time step

$$h_t = \overline{A}h_{t-1} + \overline{B}x_t$$
$$y_t = Ch_t$$

$$h_0 = \bar{B}x_0$$
$$y_0 = Ch_0 = C\bar{B}x_0$$

$$h_1 = \bar{A}h_0 + \bar{B}x_1 = \bar{A}\bar{B}x_0 + \bar{B}x_1$$
$$y_1 = Ch_1 = C(\bar{A}\bar{B}x_0 + \bar{B}x_1) = C\bar{A}\bar{B}x_0 + C\bar{B}x_1$$

$$h_2 = \bar{A}h_1 + \bar{B}x_2 = \bar{A}(\bar{A}\bar{B}x_0 + \bar{B}x_1) + \bar{B}x_2 = \bar{A}^2\bar{B}x_0 + \bar{A}\bar{B}x_1 + \bar{B}x_2$$
$$y_2 = Ch_2 = C(\bar{A}^2\bar{B}x_0 + \bar{A}\bar{B}x_1 + \bar{B}x_2) = C\bar{A}^2\bar{B}x_0 + C\bar{A}\bar{B}x_1 + C\bar{B}x_2$$

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

For inference, we are just substituting values from previous time-stamp to other and ultimately, we see a kernel.

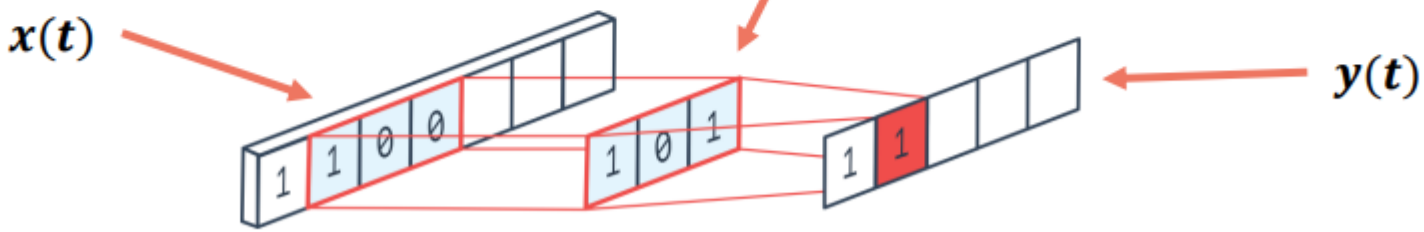
Convolutional computation

By using the formula we derived, we note something interesting: the output of the system can be calculated using a convolution of a kernel $\bar{K}$ with the input $x(t)$.

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

$$\overline{\boldsymbol{K}} = (C\overline{\boldsymbol{B}}, C\overline{\boldsymbol{AB}}, \ldots, C\overline{\boldsymbol{A}}^k\overline{\boldsymbol{B}}, \ldots) \quad (3a)$$

$$y = x * \overline{\boldsymbol{K}} \quad (3b)$$



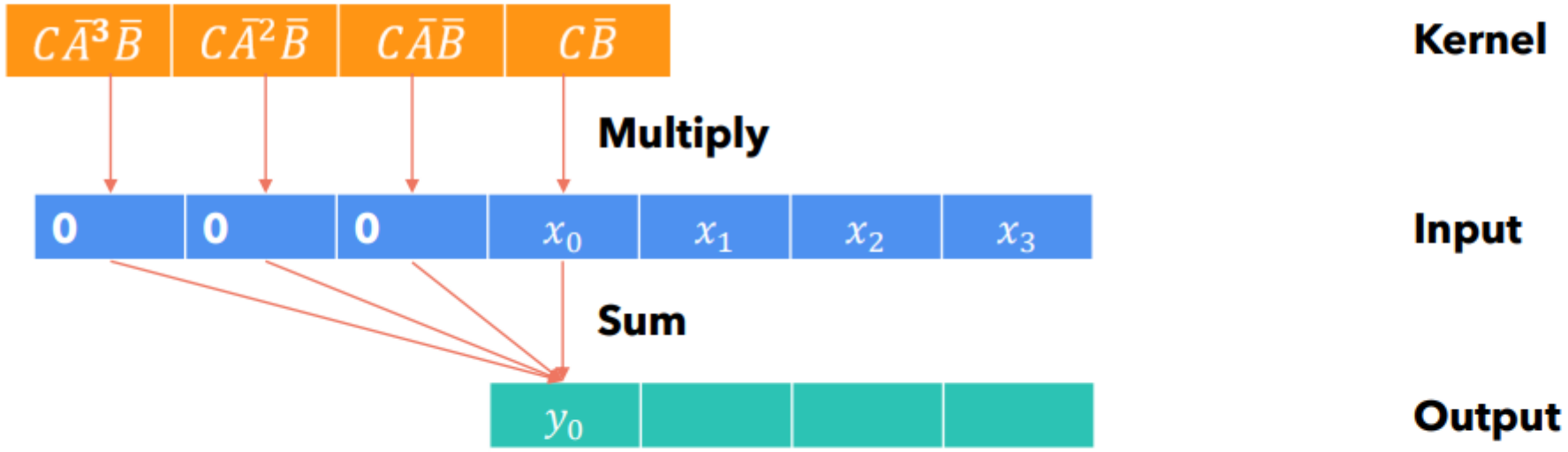
k acts as a kernel, and thus it can be used as a kernel during training for parallelization.

Convolutional formulation: step 1

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

$$\overline{\boldsymbol{K}} = (C\overline{\boldsymbol{B}}, C\overline{\boldsymbol{AB}}, \ldots, C\overline{\boldsymbol{A}}^k\overline{\boldsymbol{B}}, \ldots) \quad (3a)$$

$$y = x * \overline{\boldsymbol{K}} \quad (3b)$$



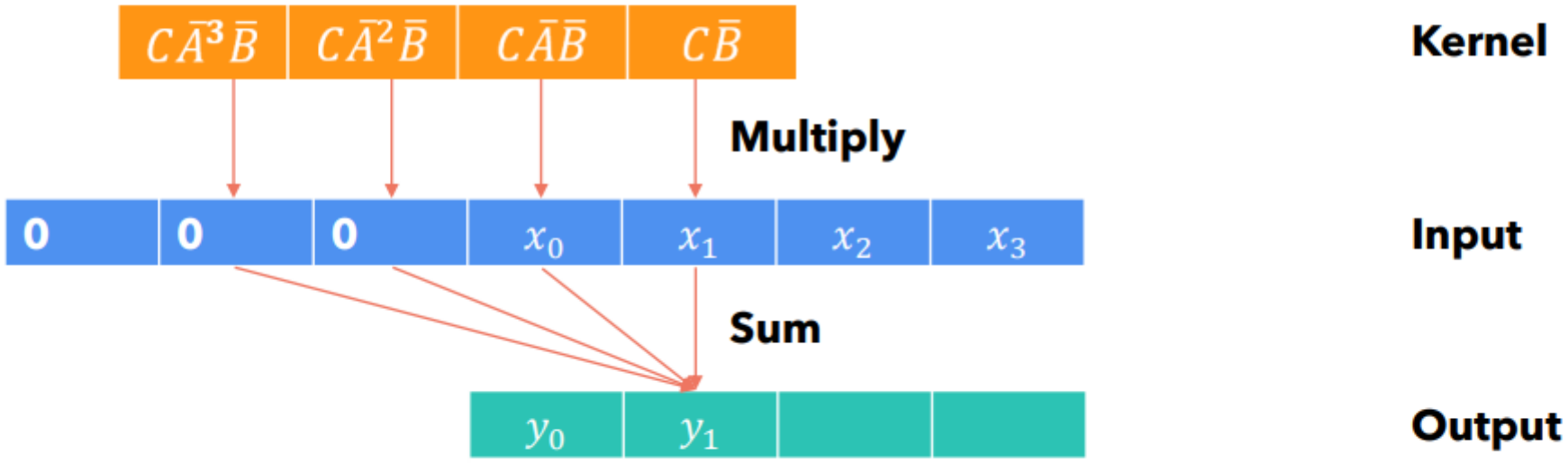
$$y_0 = C\bar{B}x_0$$

Convolutional formulation: step 2

$$y_k = C\bar{A}^k\bar{B}x_0 + C\bar{A}^{k-1}\bar{B}x_1 + \cdots + C\bar{A}\bar{B}x_{k-1} + C\bar{B}x_k$$

$$\overline{\boldsymbol{K}} = (C\overline{\boldsymbol{B}}, C\overline{\boldsymbol{AB}}, \ldots, C\overline{\boldsymbol{A}}^k\overline{\boldsymbol{B}}, \ldots) \quad (3a)$$

$$y = x * \overline{\boldsymbol{K}} \quad (3b)$$



$$y_1 = C\bar{A}\bar{B}x_0 + C\bar{B}x_1$$

Convolutional/Recurrent computation

The best thing about the convolutional calculation is that it can be parallelized,
because the output y_k does not depend on y_{k-1}, but only on the kernel and the input.

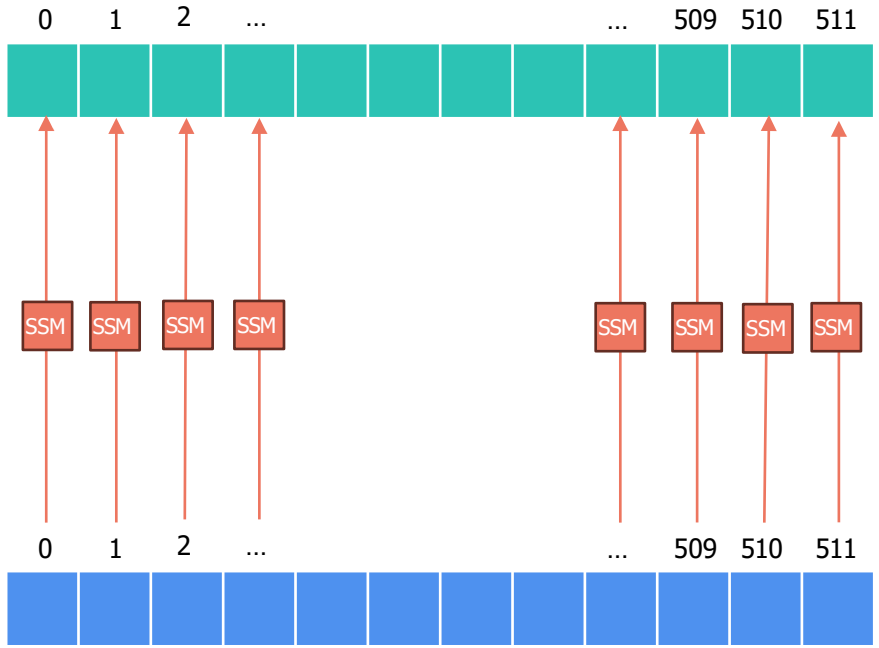
However, building the kernel can be computationally expensive, also from a memory point of view.

1. We can use the convolutional computation to perform training, because we already have all the input sequence of tokens, and it can be easily parallelized.
2. We can use the recurrent formulation to perform inference, one token at a time, using a constant amount of computation and memory for each token.

From 1 dimension to multiple dimensions

The state space model that we have studied so far, only computes one output (so only one number) for each input token (which is also represented by a single number). How can we work when the input/output signal is a vector?

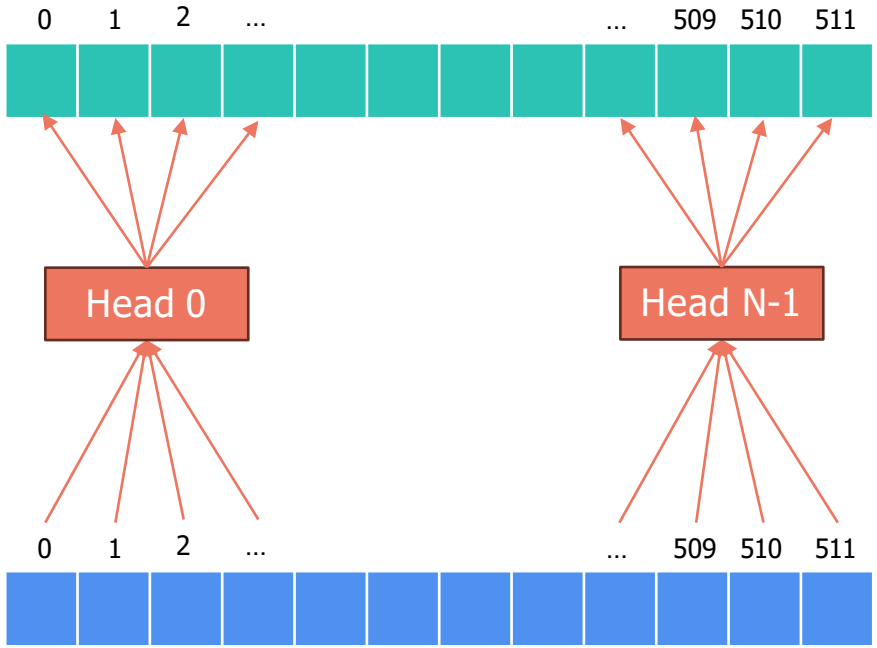
State Space Model: Each dimension is managed by an independent state space model!



Output

Input

Transformer: Each group of dimensions dimension is managed by a different head of the multi-head attention!



Of course we can parallelize all these operations by working on batches of input. This way, **the parameters A, B, C, D and the input x(t) and y(t) become vectors and matrices**.

This way, the computation will be done in parallel for all the dimensions.

The importance of the A matrix

$$h_t = \overline{A} h_{t-1} + \overline{B} x_t$$
$$y_t = C h_t$$

The A matrix in the state space model can be intuitively thought of as a matrix that “captures” information from the previous state to build the new state. It also decides how this information is copied forward in time.

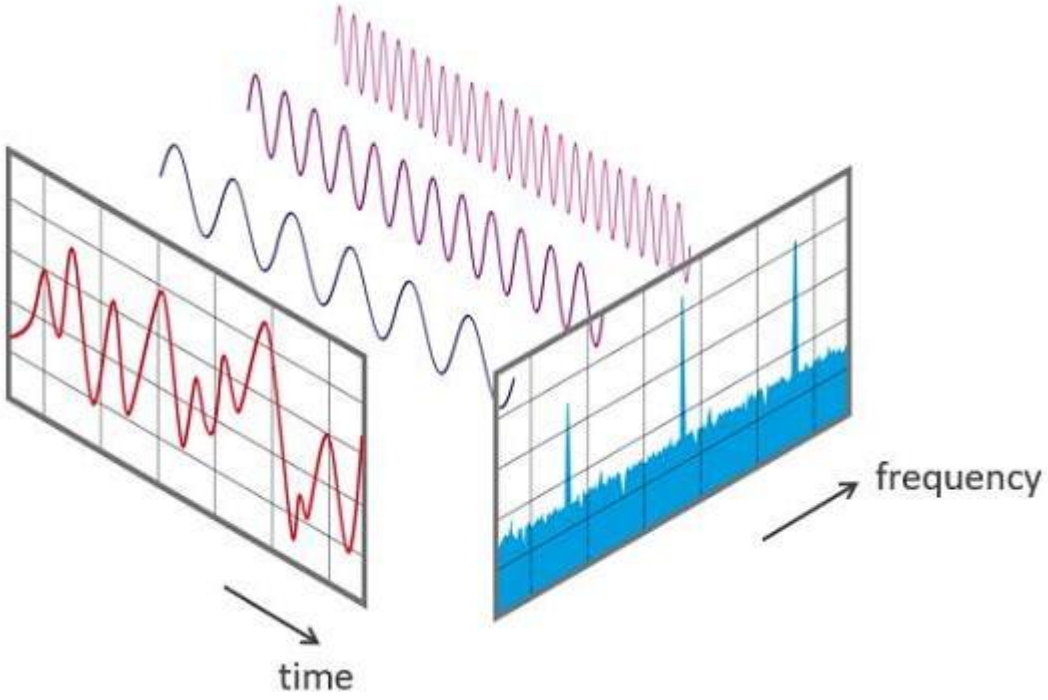
This means that we need to be careful about the structure of the A matrix, otherwise it may not capture well the history of all the inputs seen so far, which is needed to produce the next output. This is very important as the next token generated by a model should depend on the previous tokens !

To make the A matrix behave well, we use the HIPPO theory.

HIPPO : Little intuition from Fourier transformation

The Fourier transformation allows us to decompose a signal into sinusoidal functions, such that the sum of the amplitude of these functions approximates (well) the original signal.

With the HIPPO theory we do something similar, but instead of sinusoidal functions we use polynomials.



Projecting the Original Signal to construct A -

To understand how a State Space Model (SSM) captures memory, we have to look at two distinct concepts: what a **Legendre polynomial** is, and how **projection** turns a continuous stream of data into a compressed, finite set of numbers.

What is legendre polynomials ?
They are a specific family of polynomials used to approximate functions.

$$P_n(x) = \frac{1}{2^n n!} \frac{d^n}{dx^n}((x^2 - 1)^n)$$

Intuitievly, since we know the first two polynomials , we can use the **recurrence** relation to calculate others –

$$(n+1)P_{n+1}(x) = (2n+1)xP_n(x) - nP_{n-1}(x)$$

The first few polynomials look like this –

- **Degree 0:** $P_0(x) = 1$
- **Degree 1:** $P_1(x) = x$
- **Degree 2:** $P_2(x) = \frac{1}{2}(3x^2 - 1)$
- **Degree 3:** $P_3(x) = \frac{1}{2}(5x^3 - 3x)$

Why Polynomials

The most important property of Legendre polynomials is that they are orthogonal over the interval[−1, 1]. This means they are completely mathematically independent. If you multiply any two different Legendre polynomials together and integrate, the result is exactly zero.

$$\int_{-1}^{1} P_m(x)P_n(x)dx = 0 \quad \text{for } m \neq n$$

How do we project our input signal to this polynomial –

Intuition -> Think of it as casting a shadow of a 3D object onto a 2D wall.

In Mathematical terms, projecting a function f(x) onto a polynomial P(x) is done using dot product, which is simply multiplying the two functions and integrating them –

$$c_n = \frac{2n+1}{2} \int_{-1}^{1} f(x)P_n(x)dx$$

Here, c(n) is the coefficient. If you calculate these coefficients up to a certain number N, those N numbers form a state vector c(t) that acts as an incredibly accurate, compressed `sketch" of your original signal.

Turning Projections Into Memory

Step 1: Scaling Time

- In real world sequence, time doesn't sit neatly inside a fixed [-1,1]. Time keeps growing as the data arrives.
- So, HIPPO squishes all the history from [0,t] into a fixed interval so it fits the Legendre Polynomial.

Step 2: The State Vector

- The model keeps the coefficients which forms the state vector which is the vector of these coefficients after projection.

$$c(t) = \begin{bmatrix} c_0(t) \\ c_1(t) \\ \dots \\ c_{N-1}(t) \end{bmatrix}$$

- This vector is the memory. It represents the optimal reconstructions uptil time t.

Step 3: The Continous Update

- Instead of recalculating the integrals from scratch when a new data point arrives, the update is done as –

$$\frac{d}{dt}c(t) = -\frac{1}{t}Ac(t) + \frac{1}{t}Bf(t)$$

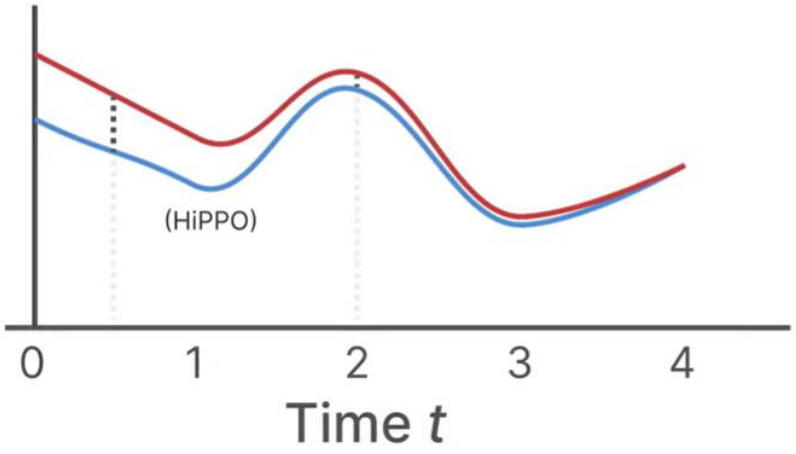
HIPPO in detail

With the HIPPO theory, we build the A matrix in such a way that it approximates all the input signal seen so far into a vector of coefficients (representing Legendre polynomials).

The difference with the Fourier transformation is that instead of building perfectly all the original signal so far, we build very precisely the more recent signal, while the older signal is decayed exponentially (like EMA). So, the state $h(t)$ captures more information about tokens seen more recently than more older ones.

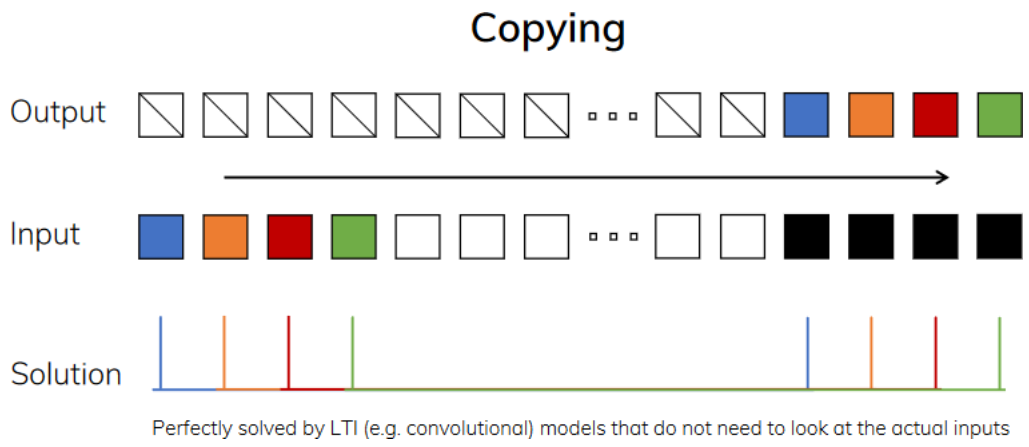
$$\textbf{(HiPPO Matrix)} \qquad \boldsymbol{A}_{nk} = -\begin{cases} (2n+1)^{1/2}(2k+1)^{1/2} & \text{if } n > k \\ n+1 & \text{if } n = k \\ 0 & \text{if } n < k \end{cases}.$$

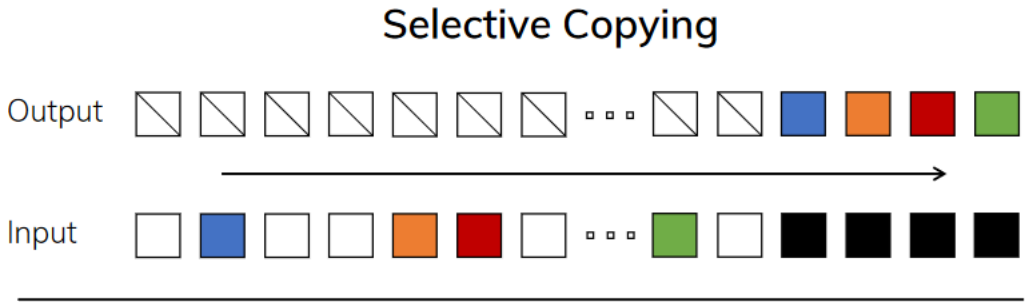
- SSM suffer from vanishing gradients exponentially as the sequence length increases because they are ordinary DE.
- HIPPO specifies a class of matrices that allows the state x(t) to memorize the history of previous all inputs using the upper triangular matrix as shown above.



SSM Limitations

The authors describe two tasks on which a vanilla SSM or even the S4 (Structured State Space Model) do not perform well, and this dictates the motivation behind mamba.





Intuition: rewrite the input one token at a time, but time-shifted.

This **can** be performed by a vanilla SSM, and the time delay can be learnt by a convolution.

Intuition: given a comment on Twitter, rewrite the comment by removing all the bad words (the white tokens).

This **cannot** be performed by a vanilla SSM, because it requires content-aware reasoning, which vanilla SSM cannot do because they are time invariant (which means the parameters A, B, C, D are the same for every token it generates).

Mamba – a Selective SSM

1. It means that parameters of the model are changed for each token.

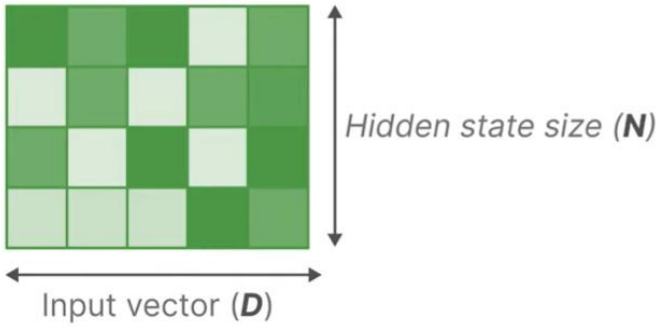
   Input : x : (B,L,D) -> We have a batch B of sequence L and each token of the sequence is of dimension D.
   Output : y : (B,L,D) -> We need ouput in the same order.

2. We modify our B matrix by a linear layer by projecting into N dimension : B(x) = linear(x)
   Note : B is orginally of D dimensions is projected to a linear space of dimension N and so we have different matrix for each token input now

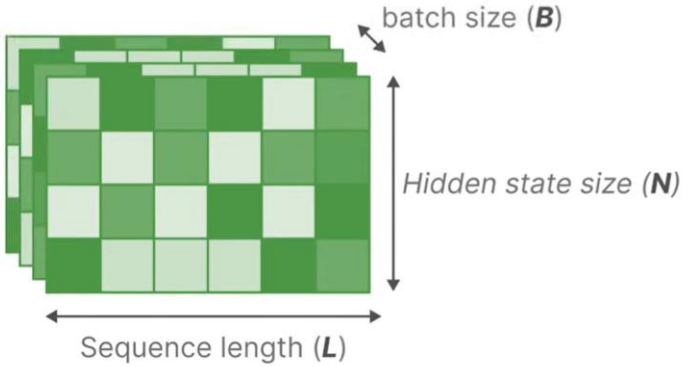
3. Same is done for C matrix and delta.

And then we further discretize it.

matrix **B** is independent of the input since
its dimensions **N** and **D** are static and do not change.



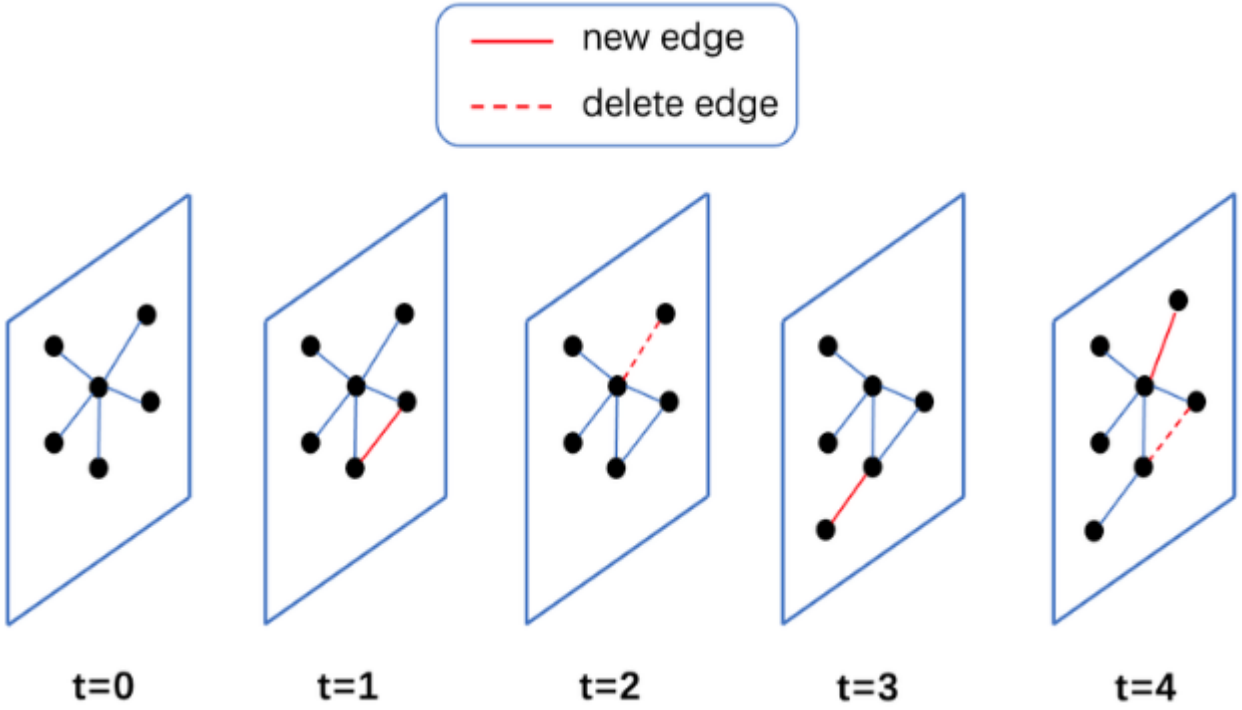
In Mamba, it instead depends on the input by incorporating
the sequence length (**L**) and batch size (**B**).



Together they choose what to keep in the state hidden layer and what not to !

What are Temporal Graphs ?

- Suppose we have a time interval [0,T]
- Temporal graphs are a sequence of static snapshots at timestamps 0,1,2,…..



Setup for a Temporal Graph

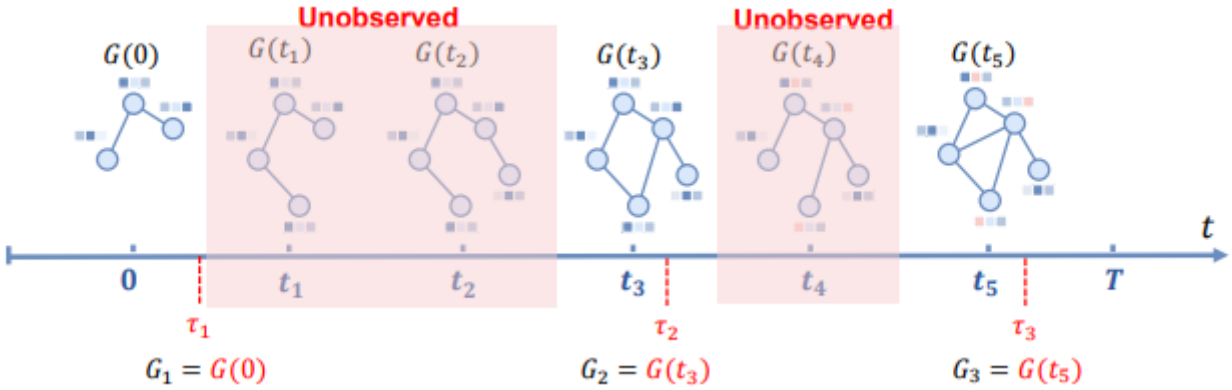
A temporal graph is characterized by two processes G and X :

- For each t ->, the process G maps t to a graph snapshot G = (V,E).
- The Edge E is defined as a sequence of events.

$$\mathscr{E} = \{\mathcal{E}_1, \ldots, \mathcal{E}_M\} \text{ with each } \mathcal{E}_m = (u_m, v_m, t_m, a_m),$$

The Process X maps ‘t’ to a node features matrix X(t) with size NxD, where N is the number of nodes and d is feature dimension.

- We observe the graph at time stamps at time stamps (mutants) -> t0, t2, t3, … with some unobserved mutations in between.



To apply the SSM layer, we want to extent the HIPPO to Temporal Graphs

- At any given time t, the goal is to find a memory representation Z(t) that best compresses the history of node feature matrix X(s) while respecting the graph's structural topology.

The loss function could be defined as –

$$\mathcal{L}_t(Z; G, X, \mu) = \int_0^t \|X(s) - Z(s)\|_2^2 \, d\mu_t(s) + \alpha \int_0^t Z(s)^\top L(s) Z(s) d\mu_t s.$$

- Here the first term aims to minimize the distance between input features X(s) and the memory state Z(s) over time interval [0,t].
- The second term is the Laplacian Regularization term which forces the memory representations of adjacent nodes in the graph to be similar.
- L(s) is the graph Laplacian at time s which represents that the memory state also preserves the structure.
- By leveraging the data from adjacent nodes, one can potentially achieve a more effective denoising effect during the process of node memory compression
- At any time s, the integrand of second term achieves its minimum value when Z(s) satisfies the structure.

Tackling the Problem of Mutations

Since we cannot see every single edge addition or deletion, the model must "estimate" the continuous evolution that happened during the unobserved time gaps.

Achieved using the mix module –

1. Feature Mixing –

- We have access to X(t-1), X(t),G(t-1) and G(t) with the rest unobserved.
- So, best possible way is to approximate the mutation information.
- Feature Mixing mixes the raw input features of the previous and current snapshot, before they enter the GNN module.

Experimental GNN Module

- The Intra – node mixing is done using a learnable function 'MIX' which effectively creates an 'interpolated' set of features that represents the unobserved mutations.
- These mixed features are then paired with the current graph topology.
- For the process of inter-node mixing, we opt to approximate the diffusion operation with a learnable shallow graph neural network (1-layer GNN) parameterized by θ

$$\widehat{X}_l^{(\mathrm{F})} = \mathrm{GNN}_\theta\left(\mathrm{MIX}_\phi\left(X_{l-1}, X_l\right), G_l\right)$$

GHIPPO

- The most favorable property of the HIPPO framework on independent inputs is that the outputs of HIPPO operators are characterized via a concise ordinary differential equation (ODE).

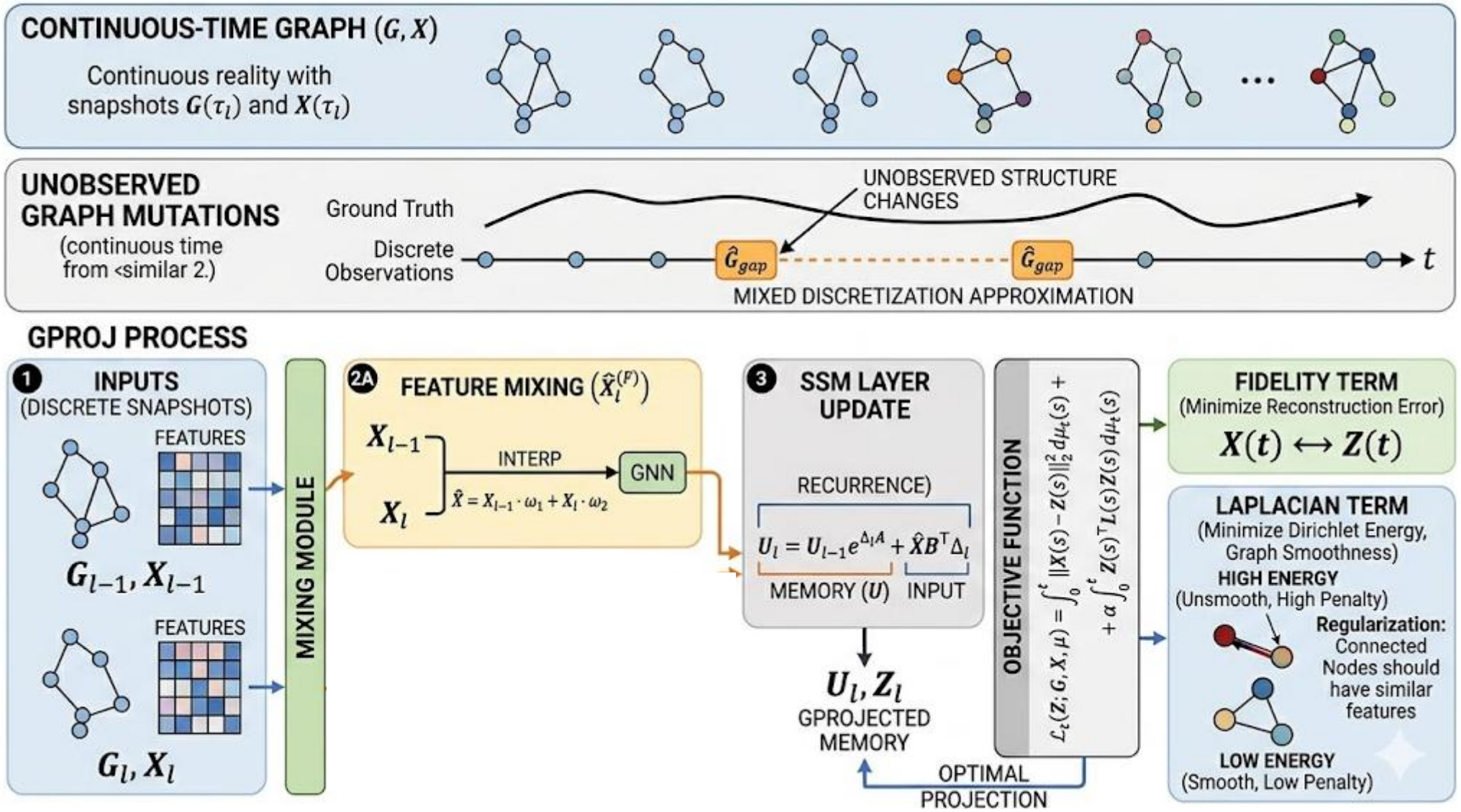
- The **GHIPPO** operator is characterized by M ODEs according to mutation times as follows :

$$\frac{dU(t)}{dt} = U(t)A^\top + (I + \alpha L(t))^{-1} X(t) B^\top, \quad 1 \leq m \leq M, t \in [t_{m-1}, t_m)$$

- Over each interval [tm, tm+1) during which the graph structure remains fixed. This property further extends to a piecewise LTI perspective over the interval [0, T].

- Intuitively, this procedure entails a sequential execution, wherein an initial diffusion operation is applied to the features of the input nodes, succeeded by an update to the memory of these nodes.

Architecture



INTERP (Interpolation): An input-dependent gating mechanism. It uses a sigmoid function to decide, for each node and feature, how much to favor the past versus the present.

Application 1 – Node Classification

DATASET	Micro – F1	Macro – F1
Transformer based (STAR)	86.70	86.26
SSM Based (Ours)	85.26	85.00

- SSM based model performed near to Transformer based model because of feature mixing and weighted average of unobserved time gaps.

About the Dataset –

- Available in XML format.
- Format : A <paper> element *contains* <author> and <title> tags.
- The dataset follows a heterogenous graph nature . Nodes are Authors, Venue, Papers, etc.

Dataset	# Nodes	# Edges	# Attributes	# Time Steps	# Classes
DBLP-3	4257	23540	100	10	3

Application 1 – Node Classification

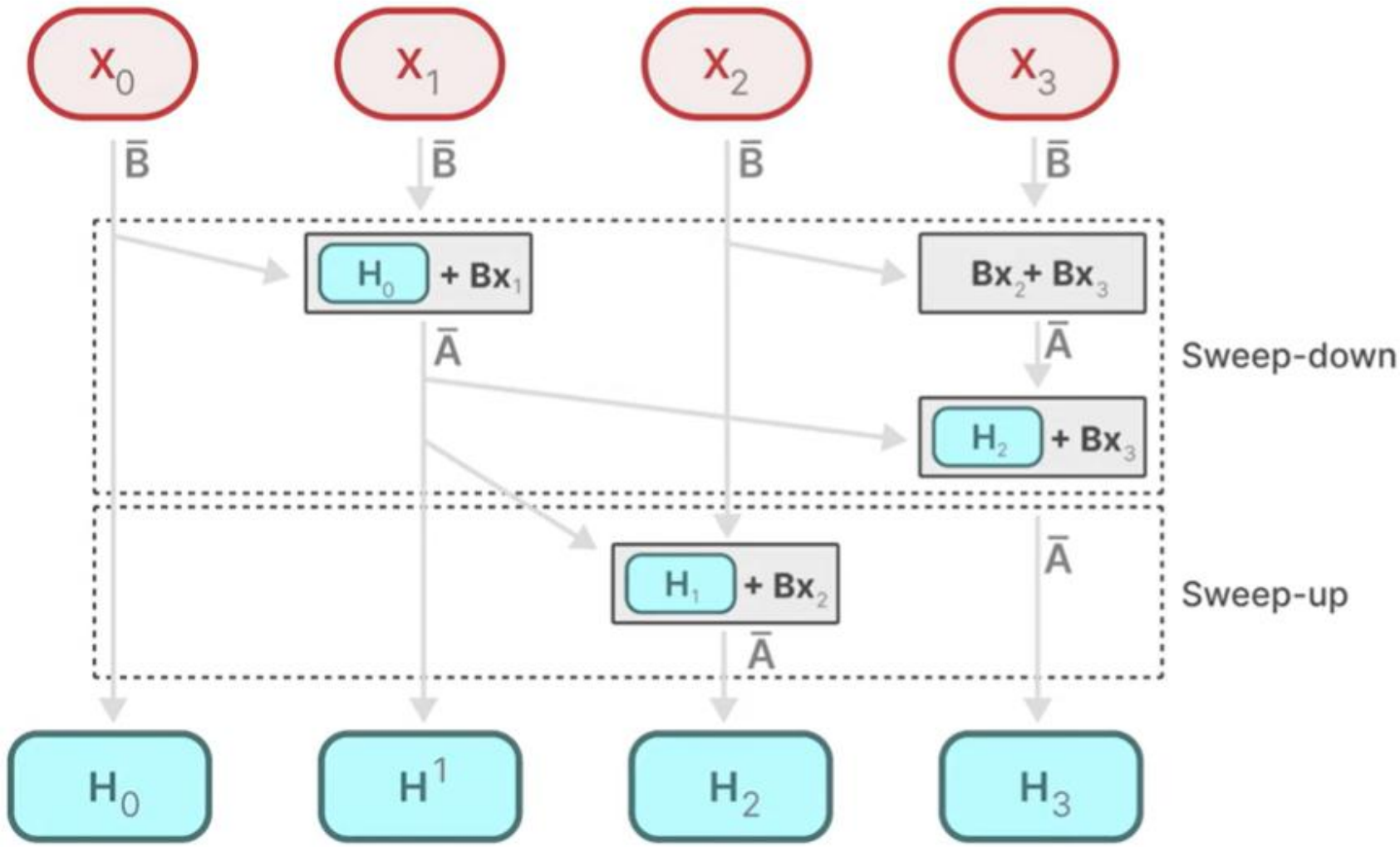
DBLP – 3 DATASET	Micro – F1	Macro – F1
Transformer based (STAR)	86.70	86.26
SSM Based (Ours)	85.26	85.00

- SSM based model performed better than transformers since brain dataset have very less number of timestamps, and very high number of edges so attention is highly affected by noise, whereas the SSM based model uses Laplacian regularizer directly into the objective function.

About the Dataset –

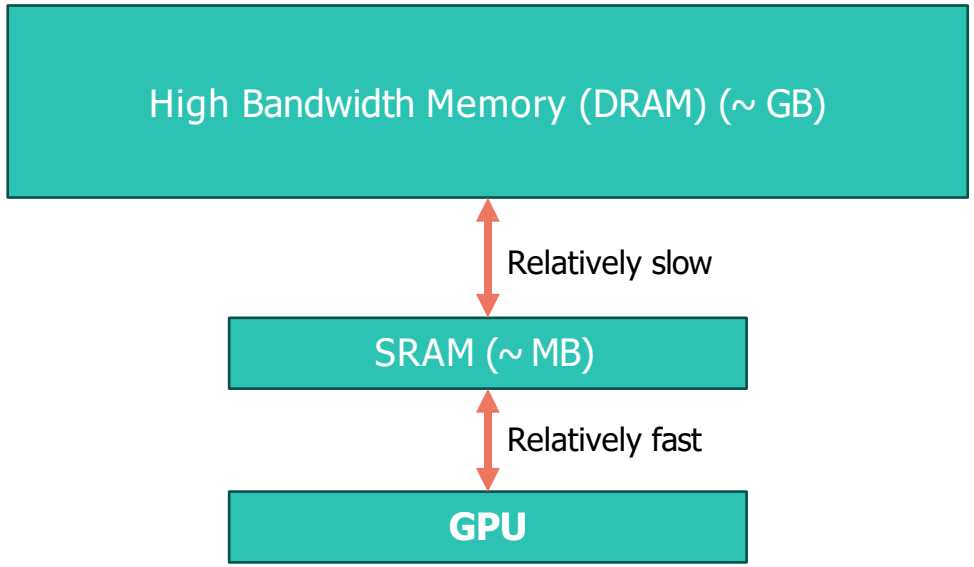
Dataset	# Nodes	# Edges	# Attributes	# Time Steps	# Classes
Brain	5000	1955488	20	12	10

This process allows for **parallel computation** of **intermediate steps**.



Parallel computation **O(n/t)**

The GPU memory hierarchy



NVIDIA A100 TENSOR CORE GPU SPECIFICATIONS (SXM4 AND PCIE FORM FACTORS)

	A100 40GB PCIe	A100 80GB PCIe	A100 40GB SXM	A100 80GB SXM
FP64	9.7 TFLOPS			
FP64 Tensor Core	19.5 TFLOPS			
FP32	19.5 TFLOPS			
Tensor Float 32 (TF32)	156 TFLOPS | 312 TFLOPS*			
BFLOAT16 Tensor Core	312 TFLOPS | 624 TFLOPS*			
FP16 Tensor Core	312 TFLOPS | 624 TFLOPS*			
INT8 Tensor Core	624 TOPS | 1248 TOPS*			
GPU Memory	40GB HBM2	80GB HBM2e	40GB HBM2	80GB HBM2e
GPU Memory Bandwidth	1,555GB/s	1,935GB/s	1,555GB/s	2,039GB/s
Max Thermal Design Power (TDP)	250W	300W	400W	400W
Multi-Instance GPU	Up to 7 MIGs @ 5GB	Up to 7 MIGs @ 10GB	Up to 7 MIGs @ 5GB	Up to 7 MIGs @ 10GB
Form Factor	PCIe		SXM	
Interconnect	NVIDIA® NVLink® Bridge for 2 GPUs: 600GB/s ** PCIe Gen4: 64GB/s		NVLink: 600GB/s PCIe Gen4: 64GB/s	
Server Options	Partner and NVIDIA-Certified Systems™ with 1-8 GPUs		NVIDIA HGX™ A100-Partner and NVIDIA-Certified Systems with 4,8, or 16 GPUs NVIDIA DGX™ A100 with 8 GPUs	

* With sparsity

** SXM4 GPUs via HGX A100 server boards; PCIe GPUs via NVLink Bridge for up to two GPUs

Because the GPU is not so fast at moving tensor around, but it's super fast at computing operations, sometimes, the problem in our algorithm may not be the number of computations we do, but how many tensors we move around in the different memory hierarchies. In this case, we say that the operation is IO-bound.

Mamba: Kernel fusion

When we perform a tensor operation, our deep learning framework (e.g. PyTorch) loads the tensor in the fast memory (SRAM) of the GPU, performs the operation (e.g. matrix multiplication), and then saves back the result in the High-Bandwidth Memory of the GPU.

What if we do multiple operations on the same tensor (e.g. 3 operations)? Then the deep learning framework would perform the following sequence:

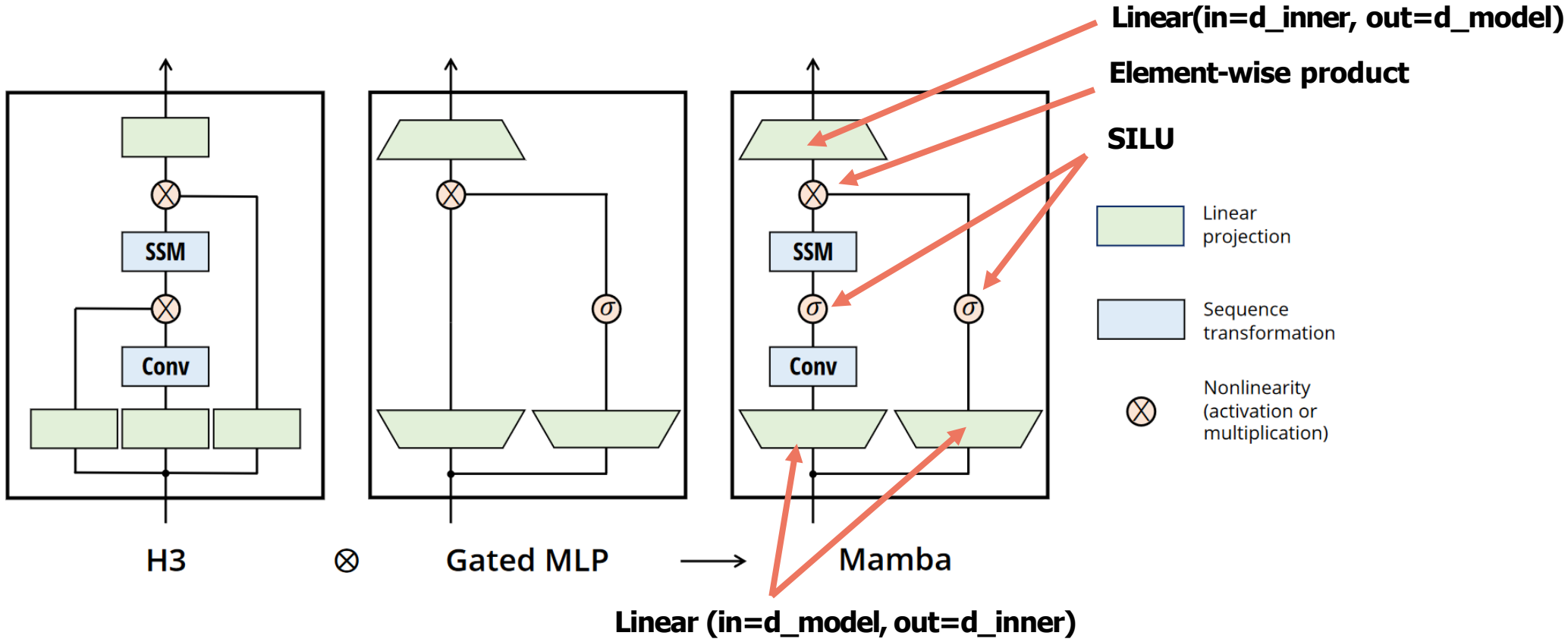
1. Load the input from HBM to SRAM, compute the first operation (CUDA kernel corresponding to the first operation) and then save back the result to HBM
2. Load the previous result from HBM to SRAM, compute the second operation (CUDA kernel corresponding to the second operation) and then save back the result to HBM
3. Load the previous result from HBM to SRAM, compute the third operation (CUDA kernel corresponding to the third operation) and then save back the result to SRAM.

As you can see, the total time is occupied by the copying operations that we are performing, since we know that GPUs are relatively slow at copying data than they are at computing operations.

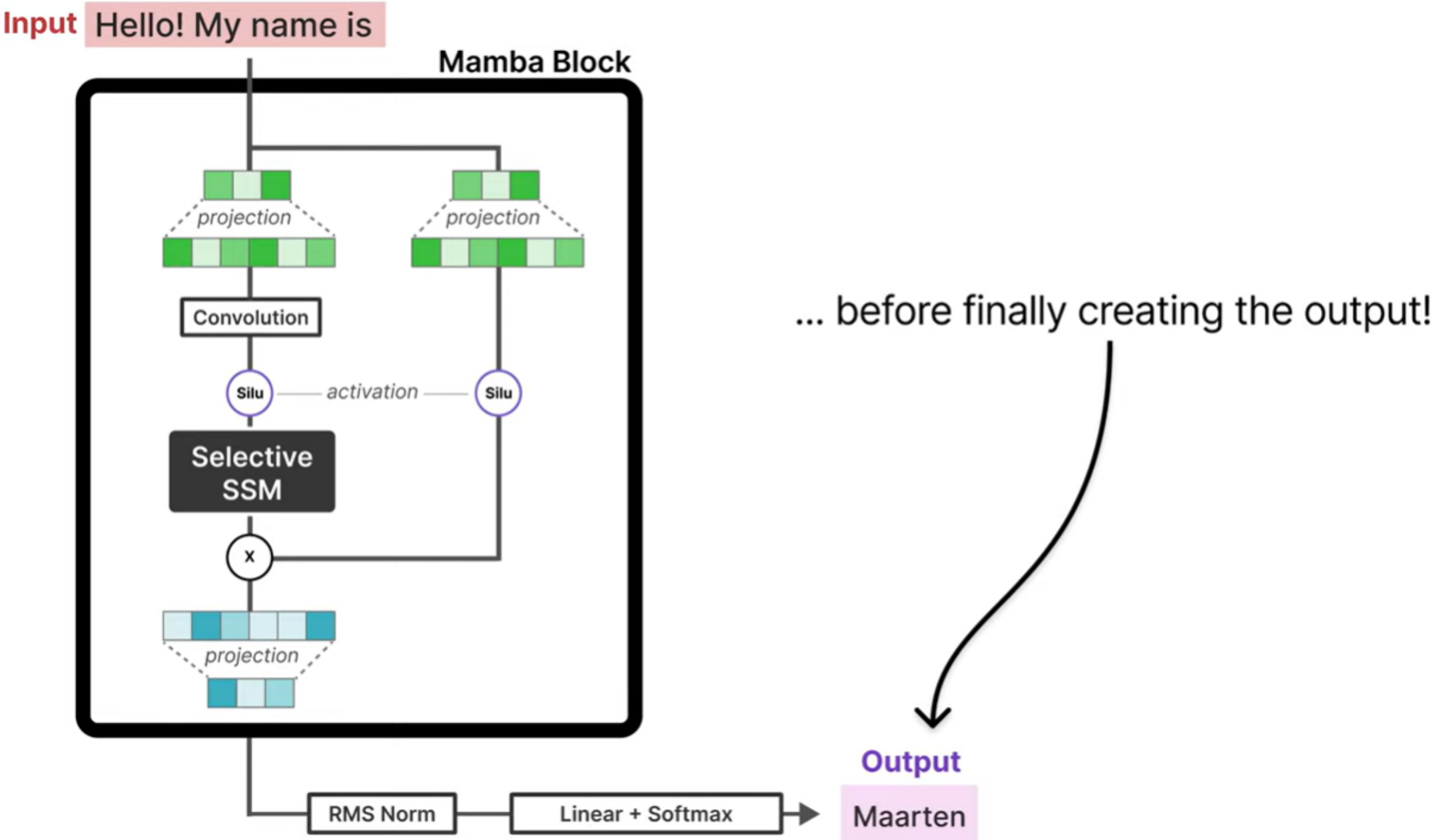
To make a sequence of operations faster, we can fuse the CUDA kernels to produce one custom CUDA kernel that does the three operations one after another without copying the intermediate results to the HBM and only copying the final results.

Mamba: the Mamba Block(1)

Mamba is built by stacking multiple layers of the Mamba block, shown below. This is very similar to the stacked layers of the Transformer model. The Mamba architecture derives from the *Hungry Hungry Hippo* (H3) architecture.



Now let's see the entire architecture of Mamba!


Input
Hello! My name is
Mamba Block
projection
projection
Convolution
Silu
activation
Silu
Selective SSM
x
projection
RMS Norm
Linear + Softmax
Output
Maarten
... before finally creating the output!

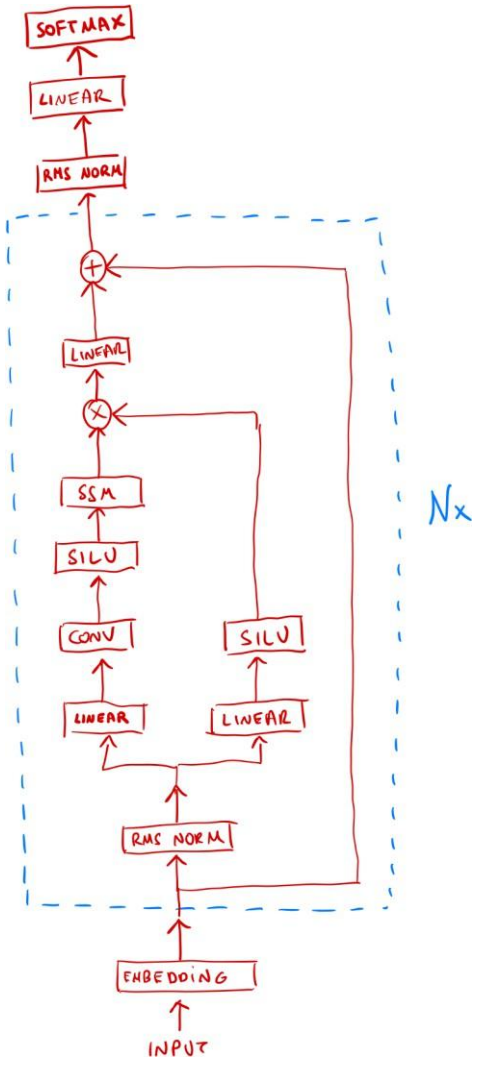
Summary of the Data Flow

In a Graph Mamba block, the process usually looks like this:

1. **Input:** Node features X.
2. **Linear Projection:** X' = XW + b (often doubling the dimension).
3. **Branching:** X' is split into a "main" path and a "gate" path.
4. **SSM Processing:** The main path undergoes 1D convolution and the Selective SSM operation.
5. **Gating:** The gate path is activated (e.g., via SiLU) and multiplied by the SSM output.
6. **Final Projection:** A final linear layer brings the dimension back down to the original size.

Mamba: the model architecture (2)

This is where we run the SSM



Mamba: the performance (1)

Model	Arch.	Layer	Acc.
S4	No gate	S4	18.3
-	No gate	S6	**97.0**
H3	H3	S4	57.0
Hyena	H3	Hyena	30.1
-	H3	S6	**99.7**
-	Mamba	S4	56.4
-	Mamba	Hyena	28.4
Mamba	Mamba	S6	**99.8**

Table 1: (**Selective Copying**.) Accuracy for combinations of architectures and inner sequence layers.

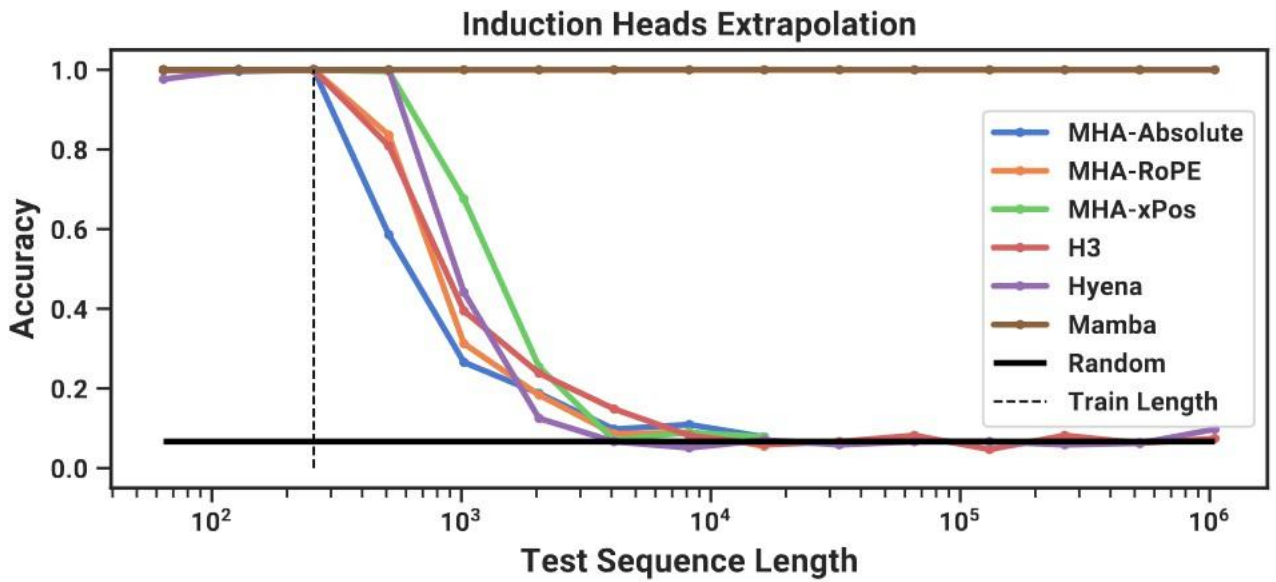


Table 2: (**Induction Heads**.) Models are trained on sequence length $2^8 = 256$, and tested on increasing sequence lengths of $2^6 = 64$ up to $2^{20} = 1048576$. Full numbers in Table 11.

Mamba: the performance (2)

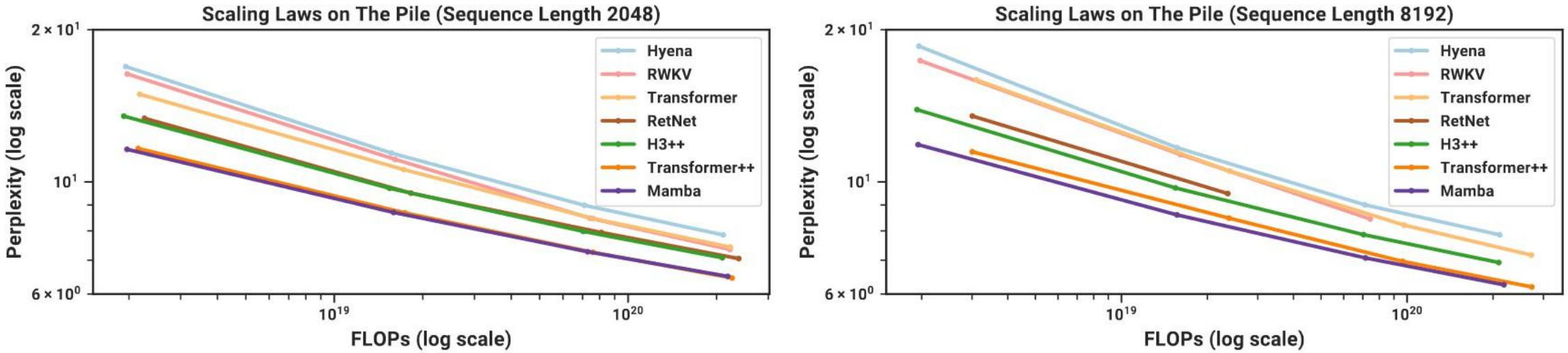


Figure 4: (**Scaling Laws.**) Models of size $\approx 125M$ to $\approx 1.3B$ parameters, trained on the Pile. Mamba scales better than all other attention-free models and is the first to match the performance of a very strong "Transformer++" recipe that has now become standard, particularly as the sequence length grows.

Thanks for watching!
Don’t forget to subscribe for more amazing content on AI and Machine Learning!